

Updated Blueprint Sections

1. Repository Structure (Updated)

hgad-cms-fraud/

├─ README.md

├─ pyproject.toml

├─ requirements.txt

├─ requirements-dev.txt

├─ .gitignore

├─ .env.example

├─ Makefile

├─ configs/

├─ data/

| └─ raw/

| └─ processed/

| └─ splits/

├─ artifacts/

| └─ graphs/

| └─ models/

| └─ embeddings/

| └─ explanations/

| └─ results/

├─ notebooks/

├─ scripts/

├─ src/

| └─ hgad_cms/

├─ tests/

├─ docs/

| └─ protocol.md

| └─ graph_schema.md

```

| └─ reproduction.md
| └─ experiment_journal.md    # ← NEW: running experiment log
| └─ failure_log.md          # ← NEW: failures, OOM, leakage, rejections
└─ paper/
    └─ tables/
    └─ figures/

```

Path	Purpose	Git tracked
docs/experiment_journal.md	Chronological record of all experiment runs, decisions, and config changes	✓
docs/failure_log.md	Record of failed runs, OOM events, leakage findings, invalid graphs, rejected decisions	✓

2. docs/experiment_journal.md Specification

Purpose

- Record every experiment run
- Capture decisions and observations
- Track configuration changes

File rules

- Append-only during the project (new entries at top or bottom — use **reverse chronological at top**)
- One entry per run_id minimum; additional entries allowed for config tweaks within the same run
- Engineer updates before starting and after completing each run
- Reviewers add an optional **Review Notes** line under Observations when validating

Template (copy into docs/experiment_journal.md)

Experiment Journal

Project: Hybrid Explainable Heterogeneous Graph Anomaly Detection for Healthcare Provider Fraud Detection

Maintainer: Implementation lead | Reviewers: validation team

Entry Template

[YYYY-MM-DD] — {Experiment Name}

Field	Value
---	---
Date	YYYY-MM-DD HH:MM
Run ID	{run_id}
Git Commit	{short_sha}
Experiment Name	{e.g., hybrid_rgc_n_fold2_v1}
Model	{e.g., R-GCN + IF + Fusion / CatBoost / ablation: no_shares_patient}
Dataset Split	{fold k / all folds / preprocess-only / graph-build-only}
Configuration	`configs/{file}.yaml` + overrides: {list or "none"}
Metrics	AUPRC={ } F1={ } Recall={ } Precision={ } ROC-AUC={ } Recall@50={ }
Observations	{what worked, what surprised, comparison to prior run}
Issues	{none / brief list}
Next Action	{e.g., tune alpha grid, retry with batch_size=256, proceed to Phase 8}
Review Notes (optional)	{reviewer sign-off or flags}

Automation hook (optional but recommended)

scripts/10_evaluate.py and each training script append a stub entry via:

```
# tracking/journal.py
```

```
def append_journal_entry(entry: JournalEntry, path: Path = Path("docs/experiment_journal.md")) ->
None
```

Manual narrative (Observations, Next Action) remains human-edited.

3. docs/failure_log.md Specification

Purpose

- Record failed experiments
- Record OOM issues
- Record leakage discoveries
- Record invalid graph constructions
- Record rejected modeling decisions

File rules

- Append on any failure — do not delete entries

- **Mandatory** when a phase gate (G1–G6) fails
- Link to Run ID and Git Commit when applicable
- Tag **Failure Type** from controlled vocabulary:

OOM | LEAKAGE | INVALID_GRAPH | GATE_FAILURE | CONVERGENCE | EXPLAINER | DATA | REJECTED_DECISION | OTHER

Template (copy into docs/failure_log.md)

Failure Log

Project: Hybrid Explainable Heterogeneous Graph Anomaly Detection for Healthcare Provider Fraud Detection

Maintainer: Implementation lead | Reviewers: validation team

Entry Template

[YYYY-MM-DD] — {Short Title}

| Field | Value |

|---|---|

| ****Date**** | YYYY-MM-DD HH:MM |

| ****Failure Type**** | {OOM / LEAKAGE / INVALID_GRAPH / GATE_FAILURE / ...} |

| ****Module**** | {e.g., hgad_cms.training.trainer / graph.builder / Phase 5 G4} |

| ****Configuration**** | `configs/{file}.yaml` — {relevant keys: batch_size, hidden_dim, fold} |

| ****Symptoms**** | {error message, metric threshold missed, gate criterion} |

| ****Root Cause**** | {diagnosed cause} |

| ****Resolution**** | {what fixed it or workaround used} |

| ****Preventive Action**** | {test added, default config change, doc update, gate checklist item} |

****Related Run ID:**** {run_id or "N/A"}

****Related Gate:**** {G1–G6 or "N/A"}

4. Journal Update Requirements — By Phase

Every phase **must** include at least one journal entry on completion. Phases with multiple runs (folds, ablations) require **one entry per run_id** or one summary entry plus fold-level metrics in **Metrics**.

Phase	When to update journal	Required fields	Minimum entries
1 — Data Pipeline	After 01_preprocess.py and after 02_create_splits.py	Experiment Name=preprocess_v1, splits_v1; Metrics=row counts, fraud rate per fold; Observations=data quality notes	2
2 — Graph Construction	After each fold graph build (or one entry if all folds identical config)	Model=graph_build; Dataset Split=fold k; Metrics=node/edge counts, GPU/RAM estimate	1–5
3 — Baselines	After each baseline family completes CV (B1–B4)	Model=baseline name; Metrics=mean±std AUPRC,F1,Recall; Observations=vs literature	4
4 — GraphSAGE	After GraphSAGE CV completes	Model=GraphSAGE+IF or encoder-only; Metrics=full set; Observations=fallback readiness	1
5 — R-GCN	After each hyperparameter config tried; minimum after final R-GCN-only CV	Model=R-GCN only; Metrics=full set; Issues=OOM if any	≥1
6 — Isolation Forest	After IF-only (B6) and after IF integrated smoke test	Model=IF_only / IF_smoke; Metrics=score distribution stats	1–2
7 — Fusion	After each α grid run; minimum after final Hybrid CV	Model=Hybrid-RGCN-IF; Metrics=full set + best α per fold; Observations=H2/H3 support	≥1
8 — Explainability	After explain pass per fold or full CV explain batch	Model=GNNExplainer; Metrics=mean fidelity, sparsity; Observations=motif notes	1
9 — Experiments	After each ablation and after significance tests	Model=ablation id; Metrics=AUPRC delta vs full; Observations=H4/H6	≥4
10 — Final Evaluation	After paper assets generated	Experiment Name=final_evaluation; Metrics=best model summary table; Next Action=paper draft	1

1. **Before run:** create entry with Date, Run ID, Git Commit, Configuration (Metrics blank).
2. **After run:** fill Metrics, Observations, Issues, Next Action.
3. **Config change:** new Run ID or explicit "Configuration delta" line — never silently rerun.
4. **Reviewer:** within 48h, add Review Notes or request failure_log entry if invalid.

5. Failure-Log Update Requirements — Phase Gate Failures

When any gate **G1–G6** fails, a failure_log.md entry is **mandatory before retry**.

Gate	Phase	Fail condition	Failure Type	Required failure_log content
G1	1	Validator fails; claim count $\neq$ 558211; fraud rate out of range	DATA or GATE_FAILURE	Symptoms=validator output; Root Cause=column/join issue; Preventive Action=validator test
G2	2	Graph build crash; NaN features; leakage unit test fail	INVALID_GRAPH or LEAKAGE	Module=graph.builder; Preventive Action=mask/leakage test
G3	3	Baseline CV incomplete; CatBoost AUPRC $\leq$ 0.50	GATE_FAILURE or CONVERGENCE	Metrics observed; Next retry config
G4	5	R-GCN OOM; val AUPRC below GraphSAGE with no fallback approval	OOM or GATE_FAILURE	OOM: batch/neighbor settings; Resolution=fallback or coarsening
G5	7	Hybrid not beating R-GCN- only on $\geq 3/5$ folds	GATE_FAILURE	Root Cause analysis; fusion α ; Preventive Action=ablation doc
G6	9–10	Missing tables; significance not run; reproducibility	GATE_FAILURE	List missing artifacts; Preventive Action=checklist item

Gate	Phase	Fail condition	Failure Type	Required failure_log content
		checklist incomplete		

Additional mandatory failure_log triggers (any phase)

Trigger	Failure Type
CUDA OOM (any module)	OOM
Provider label leakage detected	LEAKAGE
Empty or disconnected provider graph	INVALID_GRAPH
GNNExplainer fidelity < 0.50 on majority of samples	EXPLAINER
Decision to reject R-GCN for GraphSAGE fallback	REJECTED_DECISION
Invalid fold split (overlap providers)	LEAKAGE

Gate failure workflow

Gate fails

- STOP retry until failure_log entry written
- Journal entry Issues field references failure_log entry date/title
- Apply Resolution
- New Run ID for retry
- Journal entry for retry run
- Re-attempt gate

6. Updated Phase Roadmap — Journal & Failure-Log Hooks

Each phase block below adds **Documentation deliverables** only.

Phase 1: Data Pipeline

- **Documentation:** experiment_journal.md entries: preprocess_v1, splits_v1
- **On G1 fail:** failure_log.md entry required

Phase 2: Graph Construction

- **Documentation:** journal entry per fold or graph_build_all_folds_v1
- **On G2 fail:** failure_log (INVALID_GRAPH / LEAKAGE)

Phase 3: Baselines

- **Documentation:** journal entries: baseline_lr_v1, baseline_rf_v1, baseline_catboost_v1, baseline_rf_centrality_v1
- **On G3 fail:** failure_log (GATE_FAILURE)

Phase 4: GraphSAGE

- **Documentation:** journal entry: graphsage_cv_v1
- **On OOM/convergence fail:** failure_log (OOM / CONVERGENCE)

Phase 5: R-GCN

- **Documentation:** journal entry per config attempt; final rgcn_only_cv_v1
- **On G4 fail:** failure_log (OOM or GATE_FAILURE); if fallback approved → failure_log (REJECTED_DECISION for R-GCN primary) + journal Next Action = Phase 4 fallback

Phase 6: Isolation Forest

- **Documentation:** journal entries: if_only_b6_v1, if_hybrid_smoke_v1
- **On fit contamination bug:** failure_log (LEAKAGE)

Phase 7: Fusion

- **Documentation:** journal entry: hybrid_full_cv_v1 with α per fold
- **On G5 fail:** failure_log (GATE_FAILURE); journal must document H2/H3 partial outcomes

Phase 8: Explainability

- **Documentation:** journal entry: gnnexplainer_cv_v1 with fidelity summary
- **On fidelity gate fail:** failure_log (EXPLAINER)

Phase 9: Experiments

- **Documentation:** journal entries per ablation (ablation_{id}_v1); significance_tests_v1
- **On stat test anomalies:** failure_log if rerun required (OTHER)

Phase 10: Final Evaluation

- **Documentation:** journal entry: final_evaluation_v1; index linking all run_ids
- **On G6 fail:** failure_log (GATE_FAILURE) listing missing artifacts

7. Updated Reproducibility Checklist (§12 addendum)

Add to existing checklist:

- docs/experiment_journal.md contains entries for every artifacts/results/{run_id}/
- docs/failure_log.md contains entries for every gate failure and OOM event
- Each journal entry references a valid Run ID present in artifacts/results/

- Each failure_log entry with Resolution has a corresponding follow-up journal entry
- Final journal entry lists canonical run_id for paper-reported numbers

8. New Module: hgad_cms/tracking/journal.py

Purpose	Programmatic journal/failure_log append helpers
Inputs	JournalEntry / FailureEntry dataclasses
Outputs	Appended markdown sections in docs/*.md
Functions	append_journal_entry(entry) -> None append_failure_entry(entry) -> None link_failure_to_journal(failure_title, run_id) -> None
Dependencies	pathlib, datetime, gitpython optional for commit sha
Unit tests	test_journal_append_creates_file, test_failure_entry_required_fields, test_journal_id empotent_header

CLI flags:

```
python scripts/10_evaluate.py --run-id a1b2c3d4 --log-journal
```

On failure:

```
python scripts/10_evaluate.py --log-failure --type OOM --module trainer
```